Other

SWAT+ Documentation

docs.swat.tamu.edu

Contents

Pages . . . 2
Components . . . 3
HRU water balance . . . 3
Channel sediment . . . 3
Plant growth . . . 3
Outputs . . . 4
Input files . . . 4
Related pages . . . 4
Source . . . 4
Workflow . . . 4
Object types . . . 5
Re-initialization . . . 5
Related pages . . . 5
Purpose . . . 6
Source . . . 6
cal_parms.cal . . . 6
calibration.cal . . . 7
Carbon parameters . . . 8
Related files . . . 16
Source . . . 16
Two condition syntaxes . . . 16
Supported variables . . . 16
Regionalization . . . 17
Decision-table conditional updates . . . 17
Parameter dispatch . . . 17
Related pages . . . 17
Pattern . . . 17
sensitivityAPI . . . 17
Roll your own . . . 18
Related pages . . . 18
Pages . . . 18
Prerequisites . . . 19
file.cio: the index of every other input . . . 19
simulation group . . . 19
basin group . . . 20
climate group . . . 20
connect, hru, hydrology groups . . . 20
hru_parm_db, ops, lum groups . . . 21
init, soils, decision_table groups . . . 21
carbon database . . . 21
Run the model . . . 22
Read the outputs . . . 22
What to change next . . . 23
Prerequisites . . . 23

file.cio: every group used . . . 23
time.sim and print.prt . . . 24
codes.bsn . . . 24
climate . . . 24
hru.con: a single HRU . . . 24
hru-data.hru . . . 25
landuse.lum and management.sch . . . 25
aquifer.aqu and aquifer.con . . . 25
channel-lte.cha and hyd-sed-lte.cha . . . 25
Run the model . . . 25
Read the outputs . . . 25
What to change next . . . 26
What it reads . . . 26
What it draws . . . 27
How to run it . . . 27
What you should see . . . 27
Related . . . 27
Add a new HRU . . . 27
Swap the PET method . . . 28
Switch to Green-Ampt infiltration . . . 28
Change the simulation period . . . 28
Enable the carbon model . . . 29
How it is built . . . 29
When to use FORD vs this site . . . 30
Related . . . 30

SWAT+ supports two calibration modes that are configured and run from inside the model.

- **Hard calibration** fits named model parameters to observed time series (streamflow, sediment, nutrient loads). Parameter values are perturbed within absolute bounds, the model is re-run, and the simulation is compared with measured data. Implemented in `calhard_*.f90`. Driven by `cal_parms.cal` (parameter definitions and bounds) and `calibration.cal` (the change list applied each run).
- **Soft calibration** adjusts process outputs toward regional water-balance and yield targets without exposing the underlying parameters as a sample space. Targets are expressed as fractions of precipitation for the water balance, channel sediment budgets, and plant growth. Implemented in `calsoft_*.f90`. Driven by `codes.sft`, `wb_parms.sft`, `water_balance.sft`, `ch_sed_budget.sft`, `ch_sed_parms.sft`, `plant_parms.sft`, and `plant_gro.sft`.

The two modes share the same internal parameter selector (`cal_parm_select.f90`) and the same conditional change machinery (`cal_conditions.f90`).

Pages

- Hard calibration. Parameter perturbation, sources, the role of `calibration.cal`.
- Soft calibration. Regional water balance and yield targets, sources, the role of `codes.sft`.
- Parameter change files. Format of `cal_parms.cal` and `calibration.cal`, verified against the readers.
- Conditions and regionalization. Applying a change only to HRUs that match a condition (HSG, texture, plant, land use, calibration group).
- Sensitivity workflow. Driving SWAT+ from an external tool to compute sensitivity indices, using the same `cal_parms.cal` to define the sample space.

Soft calibration nudges model outputs toward regional targets expressed as fractions of precipitation (water balance), per-stream-order channel sediment budgets, and per-region plant yields. The targets are user-supplied and the model adjusts a small fixed set of parameters internally until each component falls inside its target window. Soft calibration is run separately from a normal simulation; it loops over the relevant land-use regions and process components, updates parameters, and re-runs the affected balance routine.

Components

The soft codes file (`codes.sft`) is read by `calsoft_read_codes.f90` into `cal_codes` (`type soft_calibration_codes` in `calibration_data_module.f90`). Each switch enables one component of the soft calibration:

Code	Switch	Component	Driver routine
`hyd_hru`	`n`, `a`, `b`, `y`	HRU water balance by land use in each region. `a` calibrates all components (ET, surface runoff, lateral flow, percolation). `b` calibrates baseflow and total runoff only.	`calsoft_hyd.f90`, `calsoft_hyd_bfr.f90`
`hyd_hrul`	`y/n`	`hru_lte` water balance by land use in each region	`caltsoft_hyd.f90`
`plt`	`y/n`	Plant growth (LAI, harvest index) by region	`calsoft_plant.f90`
`sed`	`y/n`	Upland sediment yield by land use in each region	`calsoft_sed.f90`
`nut`	`y/n`	HRU nutrient balance by land use in each region	(gated in `calsoft_control.f90`)
`chsed`	`y/n`	Channel widening and bank accretion by stream order	(gated in `calsoft_control.f90`)
`chnut`	`y/n`	Channel nutrient balance by stream order	(gated in `calsoft_control.f90`)
`res`	`y/n`	Reservoir budgets by reservoir	(gated in `calsoft_control.f90`)

If any switch is non-default the master flag `cal_soft` is set to `"y"` and `calsoft_control` is called instead of (or alongside) the normal simulation.

HRU water balance

`calsoft_hyd.f90` is the full water-balance loop. It iterates over land-use regions, reads the regional water-balance targets (`water_balance.sft`) and the per-parameter perturbation envelope (`wb_parms.sft`), and adjusts the active parameter set until each component is within tolerance. The components and their helper routines are:

- PET, `calsoft_hyd_bfr_pet.f90`
- ET, `calsoft_hyd_bfr_et.f90`
- Surface runoff, `calsoft_hyd_bfr_surq.f90`
- Lateral flow, `calsoft_hyd_bfr_latq.f90`
- Percolation, `calsoft_hyd_bfr_perc.f90`

The shared baseflow ratio loop is `calsoft_hyd_bfr.f90`.

Adjustable parameters for the upland water balance are read into `ls_prms` (`type soft_calib_parms`). Each entry carries a parameter name, the change type (`absval`, `abschg`, or `pctchg`), and `neg`/`pos` perturbation limits with absolute floor/ceiling values.

Channel sediment

`ch_sed_budget.sft` carries the per-stream-order sediment budget targets. `ch_sed_parms.sft` carries the perturbation envelope for channel sediment parameters, read into `ch_prms`. The channel sediment block is gated in `calsoft_control.f90` but the call to a `calsoft_chsed` driver is commented out in the current source; only the per-order accounting loop runs.

Plant growth

`plant_parms.sft` defines the plant parameter envelope (LAI potential, harvest index) and `plant_gro.sft` carries the per-region plant target table. The driver is `calsoft_plant.f90`; `calsoft_plant_zero.f90` zeros the running

statistics between iterations. After soft calibration writes the new plant parameter values, `pl_write_parms_cal` dumps a fresh `plants.plt` and a `hydrology-cal.hyd` snapshot.

Outputs

`calsoft_control.f90` writes:

- `hydrology-cal.hyd` (unit 5001) with per-HRU hydrology parameters after the run.
- A channel sediment summary (unit 5000) when `chsed = y`.

These outputs are intended to be copied back into the model inputs to lock in the soft-calibrated values.

Input files

File	Read by	Purpose
`codes.sft`	`calsoft_read_codes.f90`	Component switches.
`wb_parms.sft`	(read in `calsoft_init`)	Upland water-balance parameter envelope.
`water_balance.sft`	(read in `calsoft_init`)	Regional water-balance targets (SR, LF, PC, ET, TF as fractions of precipitation).
`ch_sed_budget.sft`	(read in `calsoft_init`)	Channel sediment budget targets by stream order.
`ch_sed_parms.sft`	(read in `calsoft_init`)	Channel sediment parameter envelope.
`plant_parms.sft`	(read in `calsoft_init`)	Plant parameter envelope.
`plant_gro.sft`	(read in `calsoft_init`)	Per-region plant yield targets.

All `*.sft` slots are declared in `input_file_module.f90`, `type input_chg`. Set the slot to `null` (or remove the file) to skip a component.

Related pages

- Hard calibration. Use after soft calibration to fit remaining parameters to observed time series.
- Parameter change files. `cal_parms.cal` defines the parameter universe and bounds shared with hard calibration.

Hard calibration applies a list of named parameter changes from `calibration.cal`, re-initializes all model objects, then re-runs the time loop. Each change names a parameter, a change type (`absval`, `abschg`, `pctchg`), a value, a list of objects (HRUs, channels, aquifers, plants, …) to apply the change to, and optional conditions. The parameter universe and the absolute min/max for each parameter are defined in `cal_parms.cal`.

Source

- Driver: `src/calhard_control.f90`
- Parameter selector: `src/cal_parm_select.f90`
- Change application with conditions: `src/cal_conditions.f90`
- Parameter database reader: `src/cal_parm_read.f90`
- Change list reader: `src/cal_parmchg_read.f90`

Workflow

1. The model is run once normally. Output baseline is captured.
2. The user edits `calibration.cal` with the parameter changes to test (or an external driver writes it).
3. SWAT+ is re-launched. During startup, `cal_parm_read` loads `cal_parms.cal` into the `cal_parms` array and `cal_parmchg_read` loads `calibration.cal` into the `cal_upd` array. Each `cal_upd` entry is crosswalked to its `cal_parms` entry by name to pick up the absolute bounds and object type.

4. Inside the simulation, `cal_conditions` is called once. For every entry in `cal_upd`, it evaluates the conditions, then walks the element list and calls `cal_parm_select` for each affected object. `cal_parm_select` does the actual write into the live model state (HRU, channel, aquifer, plant, soil layer, climate series).
5. The master switch `cal_hard = "y"` triggers a second pass: `calhard_control` calls `re_initialize` then `time_control`, re-running the full simulation with the modified parameters.

The change type drives how the value is applied to the existing parameter:

`chg_typ`	Effect
`absval`	The new value replaces the existing one.
`abschg`	The value is added to the existing one (signed).
`pctchg`	The value is interpreted as a percent change.

After every change the result is clamped to the `[absmin, absmax]` range from `cal_parms.cal`.

Object types

`cal_parms.cal` tags each parameter with an object type (`ob_typ`). The type determines which collection in the model state is iterated when the change is applied:

Code	Collection	Examples
`hru`	HRUs	`cn2`, `ovn`, `lat_ttime`, `canmx`
`hlt`	`hru_lte`	parameters of the simplified HRU
`sol`	Soil layers within an HRU (`lyr1`/`lyr2` select the range)	`awc`, `bd`, `k`
`lyr`	Soil layers	layer-targeted variants
`plt`	Plants	`lai_pot`, `harv_idx`, `epco`, `phu_mat`
`aqu`	Aquifers	`alpha`, `revap_co`, `flo_min`
`cha`, `sdc`, `rte`	Channels (legacy, degraded, routed)	channel routing and sediment parameters
`swq`	Channel water quality	nutrient parameters
`res`	Reservoirs	`res_pvol`, surface area parameters
`rdt`	Reservoir decision tables	`drawdown_days`, `withdraw_rate`
`ru`	Routing units	RU-level parameters
`bsn`	Basin	basin-wide scalars in `parameters.bsn`
`pcp`, `tmp`, `cli`	Climate gauges and series	`precip`, `temp` adjustments over date ranges
`gwf`	gwflow cells	gwflow parameters

The list above is taken from the `select case (cal_parms(ipar)%ob_typ)` block in `cal_parmchg_read.f90` and the dispatch in `cal_conditions.f90`.

Re-initialization

`calhard_control.f90` calls `re_initialize` before re-running. This resets simulation state (water content, plant state, accumulators) so the second run is comparable to the baseline rather than continuing from baseline end-state.

Related pages

- Parameter change files. Column-level format of `cal_parms.cal` and `calibration.cal`.
- Conditions and regionalization. Limiting a change to HRUs that match a condition.
- Sensitivity workflow. Driving hard calibration from an external sampler.
- Soft calibration. Adjust process outputs toward regional targets before fitting to time series.

Purpose

`cal_parms.cal` is the dictionary of every parameter that calibration can touch: its name, the object type it lives on, and the absolute range it may be moved through. `calibration.cal` is the change list applied during a hard-calibration run: each row picks a parameter from the dictionary, sets a change type and magnitude, optionally restricts the change to a list of objects, and optionally attaches conditions. Both files feed the same parameter selector (`cal_parm_select`) used by hard calibration and by parameter changes inside soft calibration.

Source

- `cal_parms.cal` reader: src/cal_parm_read.f90
- `calibration.cal` reader: src/cal_parmchg_read.f90
- Type definitions: src/calibration_data_module.f90, `type calibration_parameters` (rows of `cal_parms.cal`) and `type update_parameters` (rows of `calibration.cal`).
- File slot names: src/input_file_module.f90, `type input_chg`. Defaults: `cal_parms = "cal_parms.cal"`, `cal_upd = "calibration.cal"`.

cal_parms.cal

`cal_parm_read.f90` reads:

- Line 1: title (skipped).
- Line 2: integer `mchg_par`, the number of parameter records that follow.
- Line 3: header (skipped).
- Lines 4 .. 3 + `mchg_par`: one record per parameter, read list-directed into `cal_parms(i)`.

Record layout (`type calibration_parameters`):

#	Field	Type	Description
1	`name`	char(25)	Parameter name. Must match a case in `cal_parm_select.f90`. Examples: `cn2`, `awc`, `alpha`, `epco`, `lat_ttime`.
2	`ob_typ`	char(25)	Object type the parameter is attached to: `hru`, `hlt`, `sol`, `lyr`, `plt`, `aqu`, `cha`, `sdc`, `rte`, `swq`, `res`, `rdt`, `ru`, `bsn`, `pcp`, `tmp`, `gwf`. Determines which collection is iterated.
3	`absmin`	real	Lower bound. Any change is clamped to this value.
4	`absmax`	real	Upper bound. Any change is clamped to this value.
5	`units`	char(25)	Units string for the parameter (`mm`, `m`, `days`, `null`, …). Documentation only; the reader does not interpret it.

Example

`refdata/Osu_1hru/cal_parms.cal`:

```
cal_parms.cal: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
186
name                        obj_typ       abs_min       abs_max            units
cn2                             hru      35.00000      95.00000             null
cn3_swf                         hru       0.00000       1.00000             null
usle_p                          hru       0.00000       1.00000             null
ovn                             hru       0.01000      30.00000             null
elev                            hru       0.00000    5000.00000                m
slope                           hru       0.00010       0.90000              m/m
slope_len                       hru      10.00000     150.00000                m
lat_ttime                       hru       0.50000     180.00000             days
```

The count line on line 2 (`186`) drives the allocation of `cal_parms(0:mchg_par)`. The header on line 3 (`name`, `obj_typ`, `abs_min`, `abs_max`, `units`) is skipped; values are read by position.

calibration.cal

`cal_parmchg_read.f90` reads:

- Line 1: title (skipped).
- Line 2: integer `mcal`, the number of change records.
- Line 3: header (skipped).
- Then `mcal` change records, each followed by `conds` condition lines (often zero).

Change record

Each change record is one logical line, read list-directed in two passes. The first pass reads the fixed prefix; if the prefix indicates an object list is present, the line is backspaced and re-read to pick up the integers.

Fixed prefix:

#	Field	Type	Description
1	`name`	char(25)	Parameter name. Must match a `name` in `cal_parms.cal`. The reader crosswalks this to `num_db`.
2	`chg_typ`	char(16)	Change operator. One of `absval` (replace), `abschg` (add), `pctchg` (percent change).
3	`val`	real	Magnitude of the change, interpreted per `chg_typ`.
4	`conds`	int	Number of condition lines that follow this record. May be zero.
5	`lyr1`	int	First soil layer in range (0 means all layers). For reservoir decision tables, reused as a storage-zone flag.
6	`lyr2`	int	Last soil layer in range (0 means through last layer).
7	`year1`	int	First year (for climate parameter changes).
8	`year2`	int	Last year (for climate parameter changes).
9	`day1`	int	First day of year (for climate parameter changes).
10	`day2`	int	Last day of year (for climate parameter changes).
11	`nspu`	int	Number of object indices listed after this prefix. `0` means apply to every object of this parameter's `ob_typ`.

If `nspu > 0`, the same line continues with `nspu` integers giving the object indices to update. When `nspu == 0` the reader allocates a `num` array that lists every object of the matching type (every HRU, every channel, every plant, and so on) so the change applies globally.

Condition lines

A condition line takes one of two forms, distinguished by the literal token in column 1:

- `range <var> <val1> <val2>`. Restricts the change to objects whose `<var>` falls between `val1` and `val2`. Used in `cal_conditions.f90` for `res_pvol`.
- `<var> <alt> <targ> <targc>`. Read as a `calibration_conditions` record. Used for categorical filters: `hsg`, `texture`, `plant`, `pl_class`, `landuse`, `cal_group`, `res_typ`.

See Conditions and regionalization for the supported variable list.

Example

`refdata/Osu_1hru/calibration.cal`:

```
  hru-new.cal developed from soft data calibration
    13
 NAME           CHG_TYP                  VAL    CONDS  LYR1  LYR2   YEAR1  YEAR2   DAY1   DAY2  OBJ_TOT
cn2          pctchg                 7.714000E+00  0          0          0          0          0
petco        pctchg                -5.273950E+01  0          0          0          0          0
latq_co      pctchg                -3.732030E+01  0          0          0          0          0
alpha        pctchg                 6.859000E+00  0          0          0          0          0
```

```
epco        pctchg          6.534100E+01  0  0  0  0  0
awc         pctchg         -6.256550E+01  0  0  0  0  0
esco        pctchg          3.305500E+01  0  0  0  0  0
perco       pctchg          1.434000E+00  0  0  0  0  0
cn3_swf     pctchg         -7.710000E-02  0  0  0  0  0
canmx       pctchg         -6.680230E+01  0  0  0  0  0
lat_len     pctchg         -6.241200E+00  0  0  0  0  0
revap_co    pctchg         -5.854530E+01  0  0  0  0  0
surlag      pctchg         -2.125720E+01  0  0  0  0  0
```

In this file every record has `conds = 0` and `nspu = 0`, so each change applies to every HRU. The trailing integer count is 11 per row: `conds`, `lyr1`, `lyr2`, `year1`, `year2`, `day1`, `day2`, and `nspu` (eight integers), padded by the editor with extra zeros that the list-directed reader ignores.

Carbon parameters

SWAT+ wires 40 carbon-related parameters in `cal_parm_select.f90`. Each entry below maps a calibration name to its target field, suggests a working min/max range, and notes the object type (`bsn`, `lyr`, `plt`, `hru`) that must appear in the `ob_typ` column of `cal_parms.cal`. Bounds come from the sptAPI bundled `cal_parms.cal` (261 entries), with the temperature triple tightened to physical bounds to physically realistic ranges.

Basin scalars from `carbon.bsn` (18 entries)

name	ob_typ	units	abs_min	abs_max	description
init_seq	bsn	fraction	0.5	1.0	initial sequestration fraction (org_frac%frac_seq)
init_microb	bsn	fraction	0.0	0.1	initial microbial-humus fraction (org_frac%frac_hum_microb)
init_slow	bsn	fraction	0.2	0.6	initial slow-humus fraction (org_frac%frac_hum_slow)
init_passive	bsn	fraction	0.2	0.6	initial passive-humus fraction (org_frac%frac_hum_passive)
koc_c	bsn	none	500	1500	KOC for C transport (cb_wtr_coef%prmt_21)
solc_ratio	bsn	0..1	0.1	1.0	runoff:percolate soluble C ratio (cb_wtr_coef%prmt_44)
manure_c_frac	bsn	none	0.0	1.0	manure C partition between fresh and stable pools (man_coef%rtof)
bio_consol	bsn	none	0.05	0.5	biological consolidation factor (bio_consf)
till_consol	bsn	none	0.05	0.5	tillage consolidation factor (till_consf)
sfc_rsd_photodeg	bsn	1/day	0.0	0.05	surface residue photodegradation rate (photo_degrade_factor)
n_act_frac	bsn	fraction	0.005	0.04	fraction of organic N in active humus pool (n_act_frac)
cnr_cap	bsn	none	100	1000	upper cap on residue C:N ratio
cnr_ref	bsn	none	10	50	reference C:N ratio where decomp factor = 1
cpr_cap	bsn	none	1000	10000	upper cap on residue C:P ratio
cpr_ref	bsn	none	100	400	reference C:P ratio where decomp factor = 1
t_cbn_min	bsn	deg C	-5	5	minimum temperature bound for tmpf2 (org_con%tn); tightened to physical bounds
t_cbn_opt	bsn	deg C	25	40	optimum temperature for tmpf2 (org_con%top); tightened to physical bounds
t_cbn_max	bsn	deg C	45	55	maximum temperature bound for tmpf2 (org_con%tx); tightened to physical bounds

Note

The `min < opt < max` ordering invariant is not enforced by `cal_parms.cal` itself; calibration tools may still produce invalid combinations. The `fcgd` clamp ensures invalid orderings degrade gracefully (return 0) rather than crashing.

Per-layer fields from `carbon_lyr.bsn` (11 entries)

Each entry addresses `carbdb(ly)` or `org_allo(ly)` using the `lyr1`/`lyr2` slot of the `calibration.cal` record. Bounds are guarded to `[1, size(carbdb)]` inside the `case` branch, so out-of-range layer indices are silently no-op.

name	ob_typ	units	abs_min	abs_max	description
hp_rate	lyr	1/day	1.0e-06	1.0e-04	passive humus turnover rate (carbdb(ly)%hp_rate)
hs_rate	lyr	1/day	1.0e-05	1.0e-03	slow humus turnover rate (carbdb(ly)%hs_rate)
microb_rate	lyr	1/day	0.005	0.05	microbial biomass turnover rate (carbdb(ly)%microb_rate)
meta_rate	lyr	1/day	0.01	0.1	metabolic litter turnover rate (carbdb(ly)%meta_rate)
str_rate	lyr	1/day	0.005	0.05	structural litter turnover rate (carbdb(ly)%str_rate)
microb_top_rate	lyr	1/day	0.005	0.05	top-layer microbial activity adjustment (carbdb(ly)%microb_top_rate)
hs_hp	lyr	none	0.01	0.2	slow-to-passive humus allocation (carbdb(ly)%hs_hp)
a1co2	lyr	fraction	0.3	0.8	metabolic litter to CO2 (org_allo(ly)%a1co2)
asco2	lyr	fraction	0.3	0.8	slow humus to CO2 (org_allo(ly)%asco2)
apco2	lyr	fraction	0.3	0.8	passive humus to CO2 (org_allo(ly)%apco2)
abco2	lyr	fraction	0.0	0.8	microbial biomass to CO2 (org_allo(ly)%abco2)

Biomixing and tillage curve coefficients (6 entries)

Curve fit coefficients used by `mgt_tillfactor.f90` tillf option 4. The `zz_` prefix that the older code carried was dropped to match the column names in `carbon.bsn`.

name	ob_typ	units	abs_min	abs_max	description
bmix_a	bsn	none	1.0	5.0	biological mixing coefficient A
bmix_b	bsn	none	1.0	10.0	biological mixing coefficient B
bmix_c	bsn	none	-10.0	-1.0	biological mixing coefficient C
tillmix_a	bsn	none	1.0	5.0	tillage mixing coefficient A
tillmix_b	bsn	none	5.0	25.0	tillage mixing coefficient B
tillmix_c	bsn	none	-10.0	-1.0	tillage mixing coefficient C

Plant carbon parameters (5 entries)

Carbon-relevant parameters on the plant database (`pldb(ielem)`). These were not previously exposed even though plant biomass drives the whole NPP and residue chain.

name	ob_typ	units	abs_min	abs_max	description
bio_e	plt	(kg/ha)/(MJ/m^2)	10.0	60.0	biomass-energy ratio (drives NPP)
hvsti	plt	fraction	0.1	0.8	harvest index (controls residue vs grain C split)
rdmx	plt	m	0.5	3.0	maximum root depth
t_opt	plt	deg C	15	35	optimum plant growth temperature
t_base	plt	deg C	0	15	base plant growth temperature

The existing usle_c plant parameter is retained from earlier releases.

> **Note**
>
> For the init_* parameters to take effect, proc_cal must run **before** soil_nutcarb_init. The current main.f90 already orders these correctly; do not reorder. Previously the order was reversed and init_* calibration was a silent no-op because the pools were already sized using pre-cal values.

Related files

- carbon.bsn. Basin-wide carbon scalar inputs whose fields these parameters target.
- carbon_lyr.bsn. Per-layer carbon inputs.
- Hard calibration. How these files are consumed during a calibration run.
- Conditions and regionalization. Condition-line vocabulary.

A change in calibration.cal can carry condition lines that restrict where the change is applied. The conditions are tested object by object in cal_conditions.f90. If every condition evaluates true for a given object, the change is passed to cal_parm_select for that object; otherwise the object is skipped.

Source

- Condition evaluation: src/cal_conditions.f90
- Condition reader: src/cal_cond_read.f90
- Parameter dispatch: src/cal_parm_select.f90
- Condition record type: src/calibration_data_module.f90, type calibration_conditions and type update_conditional.

Two condition syntaxes

cal_parmchg_read.f90 peeks at the first token of each condition line:

- If the token is the literal word range, the line is read as range <var> <val1> <val2>. The condition holds when the value of <var> for the current object falls inside [val1, val2]. val1 and val2 are stored on the parent change record (shared across that record's range conditions).
- Otherwise the line is read as a calibration_conditions record: <var> <alt> <targ> <targc>. targ is a real, targc is a string. The reader stores all four fields but cal_conditions.f90 only uses var, targc, and targ depending on the variable.

Supported variables

The select case (cal_upd(ichg_par)%cond(ic)%var) block in cal_conditions.f90 accepts:

Variable	Object type	Form	Semantics
res_pvol	reservoirs	range	Reservoir principal storage volume (compared in units of 10000).
hsg	HRU soils	categorical (targc)	Hydrologic soil group must equal targc (A, B, C, D).
res_typ	reservoirs	categorical (targc)	Reservoir decision-table name must equal targc.
texture	HRU soils	categorical (targc)	Soil texture string must equal targc.
plant	HRU plant community	categorical (targc)	The HRU's plant community must contain a plant named targc.
pl_class	HRU plants	categorical (targc)	Plant class for the HRU must equal targc.
landuse	HRU land use management	categorical (targc)	hru%land_use_mgt_c must equal targc.
cal_group	HRU calibration tag	categorical (targc)	hru%cal_group must equal targc.

A change with multiple conditions implements an AND: all must hold for the object to be modified.

Regionalization

Two patterns produce a regional effect:

- **Implicit through `cal_group`.** Tag every HRU with a `cal_group` name (set on the HRU input). Then a change with a `cal_group <name>` condition runs only on that group. This is how named region-based hard calibration is typically done.
- **Explicit object list.** Put the HRU (or channel, or aquifer) indices directly on the change record (`nspu > 0`). No conditions needed. This is what soft calibration writes when it commits its perturbations.

The two can combine: an explicit object list with one or more conditions further filters the list.

Decision-table conditional updates

`cal_cond_read.f90` reads a separate file, `scen_dtl.upd`, that schedules updates by decision table. Each row gives a maximum number of times the table may fire, a table type, and the name of a `dtbl` entry in `conditional.ctl`. The reader crosswalks the table name into `dtbl_scen` and stores the result in `upd_cond` (`type update_conditional`). The same decision-table machinery used elsewhere in SWAT+ (land-use change scenarios, management triggers) is reused here.

Parameter dispatch

Once conditions pass, the change is sent to `cal_parm_select.f90` with the parameter name, change type, magnitude, absolute bounds, the target element, and the soil layer (for layer-targeted soil parameters). `cal_parm_select` is a long `select case (chg_parm)` that writes the change into the live model state via `chg_par` (the change operator) and calls the relevant re-initializer (`curno` for `cn2`, `soil_awc_init` for `awc`, and so on).

Related pages

- Parameter change files. The columns that hold these conditions on disk.
- Hard calibration. When conditions are evaluated in the calibration run.

SWAT+ has no built-in sensitivity analysis. Sensitivity is run by an external tool that writes `calibration.cal`, launches the SWAT+ executable, reads an objective value from the simulation output, and repeats. The parameter list and bounds are taken from the same `cal_parms.cal` used by hard calibration.

Pattern

1. Pick the parameters to test. Pull their absolute ranges from `cal_parms.cal` or override them with a tighter calibration range. Each parameter is one dimension of the sample space.
2. Generate a sample matrix using a sensitivity method (Sobol, FAST, Morris, LH-OAT, …). Each row of the matrix is one parameter vector.
3. For each row: write `calibration.cal` with one `pctchg` (or `absval`, `abschg`) record per parameter, run SWAT+, read the chosen objective (NSE, KGE, percent bias, total flow, …) from the output files, and record it.
4. Feed the sample matrix and the model performance vector into the sensitivity method to compute indices: first-order, total-order, second-order interaction, or elementary effects depending on the method.

The expensive step is the simulation loop. A sample size in the hundreds to low thousands is common.

sensitivityAPI

`sensitivityAPI` is a command-line tool that implements steps 2 and 4 of the pattern. It operates on a SWAT+

TxtInOut directory: it reads a parameter CSV (`par_data.stb`), writes a sample matrix (`par_sample.stb`), and after the user has run the simulations and collected performance values into `perf_report.stb`, computes sensitivity indices.

Supported methods:

Method	Sampling	Indices
`FAST`	frequency-based	S1, ST
`sobol`	Saltelli quasi-random	S1, ST, S2
`RBD_FAST`	Latin Hypercube	S1
`DMIM`	Latin Hypercube	S1 (delta)
`lhoat`	LH-OAT hybrid	S1 (elementary effects)

Usage:

```
sensitivityAPI generate_sample sobol /path/to/txtinout 1024
# run the simulations here, fill perf_report.stb
sensitivityAPI analyse_sensitivity sobol /path/to/txtinout
```

The orchestration (writing `calibration.cal` per row, launching SWAT+, parsing the output) is done by a driver such as sptAPI or spToolboX. `sensitivityAPI` is only the sampler and the post-processor.

Roll your own

Any language with a numerical library can do the same. The minimum loop:

1. Parse `cal_parms.cal` to get bounds for the parameters you care about.
2. Generate `N` parameter vectors with a Python (`SALib`), R (`sensitivity`), or in-house sampler.
3. For each vector, overwrite `calibration.cal` with the corresponding `pctchg` records and call the SWAT+ binary with the TxtInOut directory as the working directory.
4. Read the metric from `channel_sd_day.txt` (or whichever output your objective uses), record it.
5. Hand the matrix and the vector to your sensitivity routine.

Related pages

- Parameter change files. Format of `cal_parms.cal` and `calibration.cal`.
- Hard calibration. The mechanism each sensitivity run drives, one parameter vector at a time.

This section takes the small datasets that ship in `swatplus/refdata/` and walks them apart piece by piece. Each tutorial assumes you have installed SWAT+ and read the Getting Started pages.

Pages

- Ames_sub1 walkthrough. A one-subbasin, twelve-HRU watershed near Ames, Iowa, simulated 1975 to 2020. Open every input group, run, and read the outputs.
- Osu_1hru walkthrough. A single-HRU paddy near Osu, Korea, simulated 2010 to 2023. Same structure as Ames at the smallest possible scale.
- Carbon notebook. The Jupyter notebook in `swatplus/jupyter_notebooks/carbon.ipynb` that plots the soil-carbon outputs.
- Cookbook recipes. Short, copy-and-paste edits: add an HRU, switch the PET method, switch to Green-Ampt, change the simulation period, turn on the carbon model.

`Ames_sub1` is the smallest realistic SWAT+ project in the source tree. One subbasin near Ames, Iowa. Twelve HRUs. Forty-six years of daily climate (1975 to 2020). The folder is at `swatplus/refdata/Ames_sub1`.

This page opens every file in the project, in the order SWAT+ reads them, explains what is in it, and then runs the model and reads the outputs. Use it as the second pass over a SWAT+ project after the Quick start.

Prerequisites

- A working SWAT+ executable. See Install.
- A local copy of `refdata/Ames_sub1`.

file.cio: the index of every other input

`file.cio` tells SWAT+ which other files exist. It is the first file the model reads. The literal token `null` means "this slot is not used in this project".

Open `Ames_sub1/file.cio`. The relevant rows for this project are:

```
simulation        time.sim          print.prt         null              object.cnt        null
basin             codes.bsn         parameters.bsn
climate           weather-sta.cli   weather-wgn.cli   null              pcp.cli
 ↪  tmp.cli
connect           hru.con
hru               hru-data.hru
hydrology         hydrology.hyd     topography.hyd
hru_parm_db       plants.plt        fertilizer.frt    tillage.til       pesticide.pes     ...
 ↪  snow.sno
ops               harv.ops          graze.ops         irr.ops           chem_app.ops
 ↪  fire.ops   sweep.ops
lum               landuse.lum       management.sch    cntable.lum       cons_practice.lum
 ↪  ovn_table.lum
init              plant.ini         soil_plant.ini
soils             soils.sol         nutrients.sol
decision_table    lum.dtl
regions           ls_unit.ele       ...  aqu_catunit.ele
```

Most slots are `null`. Ames_sub1 has no channels, no reservoirs, no routing units, no aquifer file. It is a pure HRU water-balance run. The `connect` row points to `hru.con` only.

Reference: file.cio.

simulation group

time.sim

```
day_start  yrc_start   day_end   yrc_end      step
       0       1975         0      2020         0
```

`step = 0` means a daily time step. `day_start = 0` and `day_end = 0` mean run from January 1 of `yrc_start` to December 31 of `yrc_end`.

Reference: time.sim.

print.prt

The print configuration. The first block sets the warm-up and date window:

```
nyskip      day_start  yrc_start  day_end   yrc_end   interval
0           0               1975         0     2020          1
```

`nyskip = 0` means no warm-up years are discarded from output. The objects table that follows toggles each output file. In Ames_sub1, every row is set to `aa` only:

```
objects                  daily       monthly        yearly         avann
basin_wb                     n             n             n             y
hru_wb                       n             n             n             y
...
```

That is why the run produces `*_aa.txt` files and no daily or monthly outputs.

Reference: print.prt.

object.cnt

Counts of objects to allocate. SWAT+ reads this once and sizes its arrays accordingly.

Reference: object.cnt.

basin group

codes.bsn

Method switches that apply to the whole basin. Open `codes.bsn` and read row 3:

```
pet  event  crack  swift_out  sed_det  rte_cha  deg_cha  wq_cha  nostress  cn  c_fact  carbon
 ↪  lapse  uhyd  sed_cha  tiledrain  wtable  soil_p  gampt  atmo_dep  stor_max  i_fpwet
 ↪  gw_flow  idc_till
  2      0      0          0        0        0        0       0         0   2       0       1
 ↪  0     1        0          0       0       0      0         0         0        0        0
 ↪  3
```

Notable settings for this project:

- `pet = 2`: Hargreaves PET.
- `carbon = 1`: reserved C-FARM slot, which runs no carbon model. The dynamic carbon model now activates on `carbon = 2`; this dataset must be changed to `carbon = 2` to run it.
- `uhyd = 1`: gamma-function unit hydrograph.
- `gampt = 0`: curve-number infiltration (the default).

Reference: codes.bsn.

parameters.bsn

Numeric constants used basin-wide (sediment routing coefficients, plant-water-stress thresholds, surface-runoff lag, and so on). The defaults in this file are the same defaults a fresh QSWAT+ project produces.

Reference: parameters.bsn.

climate group

The Ames station has only two climate files: `ames.pcp` (precipitation) and `ames.tem` (temperature, legacy `.tem` extension). The `weather-sta.cli` file lists one station, `weather-wgn.cli` holds the monthly statistical generator for it, and `pcp.cli` and `tmp.cli` are index files that point at `ames.pcp` and `ames.tem` respectively.

PET is computed by Hargreaves (`pet = 2` above), so there is no PET file. Solar radiation, humidity, and wind are generated from the weather generator.

References: *.cli, *.pcp, *.tmp, *.tem.

connect, hru, hydrology groups

hru.con

The connectivity file for HRU objects. One row per HRU with its area, coordinates, elevation, and the object the HRU drains to. In Ames_sub1 every HRU drains to itself (`hyd_typ = tot` to a downstream object of `null`), so the basin output is the sum of HRU contributions with no routing.

Reference: *.con.

hru-data.hru

The pointer table. Twelve rows, one per HRU. Each row names the topography record, hydrology record, soil, land-use management, soil/plant initialization, surface storage, snow, and field for that HRU:

```
id  name      topo          hydro     soil      lu_mgt           soil_plant_init  surf_stor
 ↪  snow     field
 1  hru0001   topohru0001   hyd0001   soil_01-h1   cosy_lum      soilplant1       null
 ↪  snow001  null
 2  hru0002   topohru0002   hyd0002   soil_02      mntill_corn_lum soilplant1     null
 ↪  snow001  null
...
12  hru0012   topohru0012   hyd0012   soil_12      cosy_lum      soilplant1       null
 ↪  snow001  null
```

Eleven of the twelve HRUs are `cosy_lum` (corn-soy rotation). HRU 2 is `mntill_corn_lum` (minimum-till continuous corn). That single contrast is the point of the dataset: the other HRUs differ only in soil.

Reference: hru-data.hru.

hydrology.hyd, topography.hyd

Per-HRU hydrology and topography parameters. `hydrology.hyd` carries the curve number adjustment, lateral and tile flow coefficients, perco coefficient, ESCO, EPCO, and so on. `topography.hyd` carries slope, slope length, and field length.

hru_parm_db, ops, lum groups

These are the parameter and operation databases the HRUs and management schedules look things up in.

- `plants.plt`: plant-growth parameters per land cover code.
- `fertilizer.frt`, `tillage.til`, `pesticide.pes`, `urban.urb`, `septic.sep`, `snow.sno`: input databases.
- `harv.ops`, `graze.ops`, `irr.ops`, `chem_app.ops`, `fire.ops`, `sweep.ops`: management operation databases.
- `landuse.lum`: each row binds one `lu_mgt` name (used in `hru-data.hru`) to a CN, a management schedule, a conservation practice, and an n-table.
- `management.sch`: ordered list of operations per management name. The Ames dataset has two: `cosy_lum` (corn-soy rotation) and one continuous-corn variant.
- `cntable.lum`, `cons_practice.lum`, `ovn_table.lum`: lookups referenced by `landuse.lum`.

References: plants.plt, fertilizer.frt, tillage.til, snow.sno, *.ops, landuse.lum.

init, soils, decision_table groups

- `plant.ini` and `soil_plant.ini`: starting conditions for plant biomass, residue, soil water content, soil nutrient pools.
- `soils.sol` and `nutrients.sol`: layered soil profiles and per-layer nutrient pools for each named soil.
- `lum.dtl`: decision tables that govern automatic operations (planting, harvesting) inside the management schedule.

References: soils.sol, *.dtl.

carbon database

The dataset ships `carbon = 1` in `codes.bsn`, which is now the reserved C-FARM slot and runs no carbon model. Set `carbon = 2` to activate the dynamic CENTURY/SWAT-C model; coefficients then come from `carbon.bsn` and `carbon_lyr.bsn`, and initial carbon pools from `soil_plant.ini`.

Reference: carb_coefs.cbn.

Run the model

From inside the `Ames_sub1` folder:

```
./swatplus-<version>-linux-x86_64
```

The console will fill with `Original Simulation  mm  dd  yyyy Yr N of 46`. The full run takes a few seconds on a recent laptop.

Read the outputs

When the run finishes, the folder contains a new set of files. Start with:

simulation.out

The last lines should be:

```
Original Simulation       12  31  2020 Yr   46 of   46 Time   8:49:53

Execution successfully completed
```

If you do not see `Execution successfully completed`, the run aborted. Look at the last simulated day printed and check `diagnostics.out`.

diagnostics.out

Every line here is a missing reference. The shipped run prints:

```
DIAGNOSTICS.OUT FILE
corn_lum                CORN_rot     not found in management.sch
agrr_lum                AGRR_rot     not found in management.sch
soyb_lum                SOYB_rot     not found in management.sch
oats_lum                OATS_rot     not found in management.sch
wwht_lum                WWHT_rot     not found in management.sch
ames.tem                             file not found (tgage)
                                     file not found (basins_carbon.tes)
```

The first five lines are land-use entries in `landuse.lum` that reference schedules not present in `management.sch`. They are unused in the run (none of the HRUs use those `lu_mgt` names), so the outputs are still correct. The `ames.tem` line refers to a station gage slot in `weather-sta.cli` that was set; the file exists, but the reader did not match it to the `tgage` slot. `basins_carbon.tes` is an optional carbon test file.

Reference: Checker and diagnostics.

checker.out

One row per HRU of the resolved soil parameters:

```
sname     hydgrp  zmx(mm)  usle_k  sumfc(mm)  sumul(mm)  usle_p  usle_ls  esco   epco   cn3_swf
 ↪ perco  latq_co  tiledrain
soil_01   B       2000.0   0.0     361.5      616.18     1.0     0.3638   0.95   1.0    0.0
 ↪ 0.5    0.01     0
soil_02   B       2000.0   0.0     361.5      445.67     1.0     0.3638   0.95   1.0    0.0
 ↪ 0.5    0.01     0
...
```

In this dataset `usle_k = 0.0` on every HRU. That is a known artifact of the Ames soil definitions; sediment yield will be zero. The `zmx` is 2000 mm on every HRU; the soil rooting depth is taken from `soils.sol`. The `tiledrain` flag is `0` on every HRU.

basin_wb_aa.txt

The basin-average annual water balance. Columns are documented in Water balance outputs. A typical line for Ames_sub1:

```
precip   snofall  snomlt   surq_gen  latq   wateryld  perc   et       pet
1905.2   220.7    219.997  59.5      0.06   59.6      0.003  1846.4   3464.5
```

Precipitation is 1905 mm, ET is 1846 mm, percolation is essentially zero, and surface runoff is 60 mm. The result is dominated by ET because the basin is small and well-drained and the simulation is at HRU scale only.

Other outputs

- `hru_wb_aa.txt`: same fields as `basin_wb_aa.txt`, one row per HRU.
- `hru_nb_aa.txt`, `hru_ls_aa.txt`, `hru_pw_aa.txt`: nutrient balance, landscape balance, plant and water stress, per HRU.
- `crop_yld_aa.txt`: average annual yield by HRU and crop.
- `basin_carbon_aa.txt`, `hru_carbon_aa.txt`: carbon outputs (only when `carbon = 2` activates the carbon model).

What to change next

- Switch the PET method by editing `codes.bsn` (`pet = 0` for Priestley-Taylor, `pet = 1` for Penman-Monteith). See the cookbook.
- Turn on daily output by setting `daily` to `y` in `print.prt` for the rows you care about.
- Change `nyskip` in `print.prt` to discard the first few years as warm-up.
- Read the Carbon notebook for plotting the carbon outputs.

`Osu_1hru` is the smallest SWAT+ project that exercises every input group. One HRU, ten hectares, near Osu, Korea. The land use is `rice140_lum` (a 140-day rice rotation). The folder is at `swatplus/refdata/Osu_1hru`.

Use this dataset when you want to see every input file in `file.cio` populated, but only one HRU to track.

Prerequisites

- A working SWAT+ executable. See Install.
- A local copy of `refdata/Osu_1hru`.

file.cio: every group used

Unlike Ames_sub1, this project populates almost every slot. Open `Osu_1hru/file.cio`:

```
simulation        time.sim          print.prt         null              object.cnt
basin             codes.bsn         parameters.bsn
climate           weather-sta.cli   weather-wgn.cli   null              pcp.cli
 ↪  tmp.cli           slr.cli           null              wnd.cli
connect           hru.con           null              rout_unit.con     null
 ↪  aquifer.con       null              null              null      recall.con
 ↪  null              null              null              chandeg.con
channel           initial.cha       null              null              null
 ↪  nutrients.cha     channel-lte.cha   hyd-sed-lte.cha
reservoir         initial.res       null              null              sediment.res
 ↪  nutrients.res     weir.res          wetland.wet       hydrology.wet
routing_unit      rout_unit.def     rout_unit.ele     rout_unit.rtu
hru               hru-data.hru
aquifer           initial.aqu       aquifer.aqu
hydrology         hydrology.hyd     topography.hyd    field.fld
```

The project has channels, aquifers, routing units, reservoirs, recall, and a wetland. Most of those tables are populated with template entries from the SWAT+ Editor; the single HRU in `hru.con` is what actually runs.

Reference: file.cio.

time.sim and print.prt

`time.sim`:

```
day_start  yrc_start   day_end   yrc_end      step
        1       2010       365      2023         0
```

January 1 2010 through December 31 2023, daily.

`print.prt` has `nyskip = 3`, so the first three years are discarded. The objects table sets `basin_wb` to `yearly` and `avann`. Every other object is `avann` only.

References: time.sim, print.prt.

codes.bsn

```
pet  event  crack  rtu_wq  sed_det  rte_cha  deg_cha  wq_cha  nostress  cn  c_fact  carbon
 ↪  lapse  uhyd  sed_cha  tiledrain  wtable  soil_p  gampt  atmo_dep  stor_max  i_fpwet
  1      0      0       1         0        0       0       1         0   0       0       0
 ↪  0     1        0          0       0       0      0          a        0        0
```

Settings worth noting:

- `pet = 1`: Penman-Monteith PET. Needed because the climate has solar radiation, humidity, and wind.
- `rtu_wq = 1`: routing-unit water quality on.
- `wq_cha = 1`: in-stream water quality on.
- `carbon = 0`: legacy static mineralization (no C-FARM, no Century).
- `gampt = 0`: curve-number infiltration.

Reference: codes.bsn.

climate

Five climate files for the Imsil station:

- `Imsilpcp.pcp`: daily precipitation.
- `Imsiltmp.tmp`: daily max and min temperature.
- `Imsilsol.slr`: daily solar radiation.
- `Imsilwind.wnd`: daily wind speed.

(Relative humidity is generated from the weather generator.) The index files `pcp.cli`, `tmp.cli`, `slr.cli`, `wnd.cli` each list one filename, and `weather-sta.cli` binds those four index files to the station `s35610n127290e`.

References: *.cli, *.pcp, *.tmp.

hru.con: a single HRU

```
id  name     gis_id  area  lat       lon         elev    hru  wst              cst  ovfl  rule
 ↪  out_tot
 1  hru0001       1    10  35.65311  127.36763   350.97   1   s35610n127290e   0    0     0
 ↪  0
```

Area is 10 ha. `hru = 1` and `wst = "s35610n127290e"` bind this row to HRU 1 and to the weather station. `out_tot = 0` means no downstream routing object: the run produces HRU-scale outputs only.

Reference: *.con.

hru-data.hru

The single HRU points at:

```
id  name      topo          hydro      soil        lu_mgt        soil_plant_init  surf_stor
 ↪  snow     field
 1  hru0001   topohru0001   hyd0001    PadHOEGOG   rice140_lum   soilplant1       paddy0001
 ↪  snow001  null
```

- `soil = PadHOEGOG`: a paddy soil. See `soils.sol`.
- `lu_mgt = rice140_lum`: maps in `landuse.lum` to plant community `rice140_comm` and management schedule `rice140_rot`.
- `surf_stor = paddy0001`: surface-storage record for ponded water. This is what makes it a paddy.

Reference: hru-data.hru.

landuse.lum and management.sch

`landuse.lum` row for `rice140_lum` binds the HRU to:

- `plnt_com = rice140_comm` (plant community in `plant.ini`).
- `mgt = rice140_rot` (management schedule in `management.sch`).
- `cn2 = rc_strow_g` (curve-number table entry in `cntable.lum`).
- `cons_prac = cross_slope`.
- `ov_mann = convtill_res`.

`management.sch` then describes the 140-day rice rotation: paddy-fill, transplant, fertilize, harvest, drain.

Reference: landuse.lum.

aquifer.aqu and aquifer.con

`aquifer.aqu` defines twenty shallow aquifer records (`aqu011` through `aqu102`) plus one deep aquifer (`aqu_deep010`). They share identical parameters. The single HRU links to one of them through `aquifer.con` so the percolation has somewhere to go.

Reference: *.aqu.

channel-lte.cha and hyd-sed-lte.cha

A template channel table is present so the file structure validates, but with `out_tot = 0` on the HRU, no flow reaches a channel and the channel outputs will be zero.

Reference: *.cha.

Run the model

```
cd path/to/Osu_1hru
./swatplus-<version>-linux-x86_64
```

The run takes a few seconds because there is only one HRU. Watch the console for `Original Simulation` lines.

Read the outputs

simulation.out

The last line should be `Execution successfully completed`.

diagnostics.out

Every line is a missing reference. Expect a long list because the Editor wrote many template channel and aquifer rows whose companion property records do not all exist. The HRU run itself is unaffected.

Reference: Checker and diagnostics.

checker.out

A single row for `PadHOEGOG`. Confirm `zmx`, `sumfc`, `sumul`, `hydgrp` are reasonable for a paddy.

basin_wb_aa.txt and basin_wb_yr.txt

Annual and average-annual water balance for the basin. Because there is one HRU, basin equals HRU. Compare `basin_wb_yr.txt` (per-year values) to `basin_wb_aa.txt` (the average over years 4 through 14, since `nyskip = 3`).

hru_wb_aa.txt

The water-balance row for HRU 1. Look at `irr` (irrigation depth, large for a paddy), `surq_gen` (surface runoff), `et`, `perc`.

Plant outputs

`crop_yld_aa.txt` reports rice yields by HRU and crop. `hru_pw_aa.txt` carries plant biomass and water-stress days.

What to change next

- Switch to Hargreaves PET (`pet = 2` in `codes.bsn`). The model will stop reading the solar, humidity, and wind files.
- Turn on daily output for `basin_wb` to inspect the paddy fill and drain pattern through the season.
- Change the simulation period in `time.sim` to a single year for a fast iteration loop.

See the cookbook for these edits.

The SWAT+ source tree ships one analysis notebook: `jupyter_notebooks/carbon.ipynb`. It reads the carbon-output files for a single HRU and draws time-series plots of every carbon pool the C-FARM / Century model tracks. Use it after running a project with the carbon model enabled.

What it reads

The notebook reads the following files from a SWAT+ output folder:

- `hru-data.hru` and `landuse.lum`. Used to look up the soil name, the `lu_mgt` name, and the management identifier for a chosen HRU.
- `mgt_out.txt`. Operation log: planting, harvest, tillage, fertilization. The notebook overlays these as event markers on every plot.
- `hru_cbn_lyr.txt`. Total soil carbon per soil layer.
- `hru_seq_lyr.txt`. Sequestered carbon per soil layer.
- `hru_rsdc_stat.txt`. Surface residue and soil carbon pools.
- `hru_plc_stat.txt`. Plant carbon pools.
- `hru_cpool_stat.txt`. Residue and soil carbon pools by layer (metabolic, structural, lignin, microbial, slow humus, passive humus).
- `hru_org_trans_vars.txt`. Organic-matter transformation variables.
- `hru_cflux_stat.txt`. Carbon flux statistics including CO2.

The notebook expects these to be produced by a run where `print.prt` lists each of those output rows with at least one frequency turned on. Daily output is what the notebook is designed for.

What it draws

The notebook is organised by output file. Each section opens with an "Initialize" cell that loads the file into a DataFrame, then one or more graph cells produce a Plotly figure. Sections, in order:

1. Total carbon by layer depth (`hru_cbn_lyr`).
2. Total sequestered carbon and 300 mm sequestered (`hru_seq_lyr`).
3. Surface residue and soil carbon pools at 10 mm depth (`hru_rsdc_stat`).
4. Soil carbon pools at a user-specified depth.
5. Live root mass by soil layer.
6. Plant stats from `hru_plc_stat`.
7. Residue and soil carbon pools (`hru_cpool_stat`) at 10 mm.
8. Same pools at a user-specified depth.
9. Soil residue, metabolic, structural, lignin, microbial, slow humus, and passive humus pools by soil layer.
10. Soil water content by soil layer.
11. Organic-matter transformation variables (`hru_org_trans_vars`).
12. Carbon fluxes (`hru_cflux_stat`), non-CO2 and CO2.

Every plot shows the chosen variable as a time series. PLANT, HARVEST, and TILLAGE events from `mgt_out.txt` are overlaid as vertical markers so you can read the management context off the figure.

How to run it

1. Run SWAT+ on a project with the carbon model on. In `codes.bsn`, set `carbon = 2` to activate the dynamic CENTURY/SWAT-C model (`carbon = 1` is the reserved C-FARM slot and runs no carbon model). The `Ames_sub1` reference dataset ships `carbon = 1`, so change it to `carbon = 2`.
2. Turn on the relevant outputs in `print.prt`. At minimum, `hru_cb` and the carbon-pool outputs need `daily` set to `y`.
3. Open `carbon.ipynb` in JupyterLab or VS Code.
4. Edit the first code cell. Set:
   - `hru_id` to the HRU you want to plot.
   - `wdir` and `sdir` so that `f"{wdir}/{sdir}"` is the folder that holds the SWAT+ outputs.
5. Run all cells (Run All).

The notebook uses `pandas`, `plotly`, and the standard library. Install with:

```
pip install pandas plotly
```

What you should see

Each graph cell renders a Plotly figure inline. Event markers from `mgt_out.txt` line up the plots with the management schedule. The "Initialize" cells print "File not found" and stop rendering that section if the corresponding output file is missing; this is expected when an output is not enabled in `print.prt`.

Related

- Theory: Carbon covers what the pools and fluxes represent.
- print.prt for the output switches.
- Cookbook: enabling the carbon model for the input edits.

Five common edits to a working SWAT+ project. Each recipe lists the file to open, the line to change, and what to check after running. Every recipe assumes you start from a project that already runs successfully.

Add a new HRU

You need a new entry in `hru.con`, `hru-data.hru`, and a row in any companion table the new HRU points to that does not yet contain the records (`topography.hyd`, `hydrology.hyd`, `soils.sol`, …).

1. Open `hru.con`. Append a new row, copying an existing row and bumping the `id`. Edit `area`, `lat`, `lon`, `elev`, and `hru` (the HRU index). Keep `wst` pointed at an existing weather station.
2. Open `hru-data.hru`. Append a row with the same `id` and a new `name`. Point its `topo`, `hydro`, `soil`, `lu_mgt`, `soil_plant_init`, `surf_stor`, `snow`, `field` columns at names that exist in the corresponding tables.
3. Open `object.cnt` and update the `hru` count and `obj` count to reflect the new total.
4. Run. After the run, check `checker.out` for a new row matching the soil name. If `usle_k`, `zmx`, `sumfc` look wrong, your `soils.sol` row is incomplete.

References: hru-data.hru, *.con, object.cnt.

Swap the PET method

`codes.bsn`, field 3 (`pet`):

- `0`: Priestley-Taylor.
- `1`: Penman-Monteith.
- `2`: Hargreaves.
- `3`: read from `pet.cli`.

Edit the value, save, run.

Penman-Monteith needs solar radiation, relative humidity, and wind. If those climate files are absent, the weather generator will fill them in, and the result will not match a station-driven Penman-Monteith run.

Hargreaves needs only minimum and maximum temperature. It is the right choice for sites with minimal climate data.

If you set `pet = 3`, also point `pet_file` (field 1 of `codes.bsn`) at a valid `pet.cli` and provide the daily-PET file it references.

Reference: codes.bsn.

Switch to Green-Ampt infiltration

`codes.bsn`, field 21 (`gampt`):

- `0`: curve-number infiltration (default).
- `1`: Green-Ampt with Mein-Larson.

Green-Ampt runs on a sub-daily time step. Set `step` in `time.sim` to a non-zero value (the source supports `step = 24` for hourly). Your precipitation file must contain sub-daily values; a daily `.pcp` will not be disaggregated for you.

After the run, look at `basin_wb_aa.txt`. The split between `surq_gen` and `perc` shifts when the method changes; if it does not, the run is still using curve number.

References: codes.bsn, time.sim.

Change the simulation period

`time.sim`:

```
day_start  yrc_start  day_end  yrc_end  step
        0       1975        0     2020     0
```

- `day_start = 0`: start January 1 of `yrc_start`.
- `day_start = N > 0`: start on Julian day N of `yrc_start`.
- `day_end = 0`: end December 31 of `yrc_end`.
- `day_end = N > 0`: end on Julian day N of `yrc_end`.

To shorten a run, change `yrc_start` and `yrc_end`. To set a warm-up that is excluded from the average-annual outputs, leave `time.sim` alone and set `nyskip` in `print.prt`.

Your climate files must contain data for every day of the simulation period. If they do not, SWAT+ writes a "file not found" or runs off the end of the file.

References: time.sim, print.prt.

Enable the carbon model

`codes.bsn`, field 14 (`carbon`):

- `0`: static (legacy mineralization).
- `1`: C-FARM.
- `2`: Century.

For `1` or `2`, also:

1. Provide a `carb_coefs.cbn` file in the project folder. The Ames_sub1 dataset ships one you can copy. If `cswat /= 0` and no `carb_coefs.cbn` is present, SWAT+ uses internal defaults and writes a notice to `diagnostics.out`.
2. Confirm `soil_plant.ini` (or `soils.sol`, depending on tool version) carries initial organic carbon by layer. Without realistic initial pools, the first decade of output is transient as pools spin up.
3. Turn on carbon outputs in `print.prt`. Set `hru_cb` and the per-pool outputs (`hru_cbn_lyr`, `hru_cpool_stat`, `hru_cflux_stat`, …) to `daily` if you want to use the carbon notebook.

After the run, `basin_carbon_aa.txt`, `hru_carbon_aa.txt`, and the per-pool files appear in the output folder. If they are missing, the print switches were not on.

References: codes.bsn, carb_coefs.cbn, print.prt.

The authoritative API reference for the SWAT+ Fortran source is generated by FORD and published at:

https://swat-model.github.io/swatplus

That site contains:

- Every module, derived type, subroutine, and function declared in `swatplus/src/`.
- Call and caller graphs.
- Source-listing pages for each `.f90` file.
- The leading comment block of every routine, indexed and searchable.

FORD reads the Fortran source directly. Whatever is in the source is reflected on the site; the docs do not duplicate this information.

How it is built

The FORD configuration lives in `ford.md` in the root of the SWAT+ source repository. Excerpt:

```
project: SWAT+
output_dir: /drive/c/repositories/swatplus/build/doc
src_dir: src
version: <git describe>
graph: true
coloured_edges: true
```

`ford.md.in` is the template; CMake substitutes the version from `git describe` and writes the resolved `ford.md`. The build command is:

```
ford ford.md
```

The resulting HTML tree is published from the `gh-pages` branch by a GitHub Action.

When to use FORD vs this site

- Use **this site** for the input file formats, output file formats, theory, calibration workflow, and how to run the model.
- Use **the FORD site** when you need to know what a specific subroutine does, what types a routine reads and writes, or where in the call tree a given variable is touched. It is the right place to start before modifying SWAT+ source.

Related

- Developer guide: how to clone, build, test, and contribute.
- Coding conventions: the style FORD picks up from comments.